CHAPTER 11
영상 처리 응용 사례 I

PART 03 영상처리 응용 사례

11.2 2차원 히스토그램을 이용한 이미지 검색

- ◆11.2.1 2차원 히스토그램
- ◆11.2.2 검색 대상 영상 히스토그램 계산 및 저장
- ◆11.2.3 유사도 비교
- ◆11.2.3 비교 결과 영상 출력
- ◆11.2.4 전체 프로그램

11.2 하르 분류기를 이용한 얼굴검출 및 성별 분류

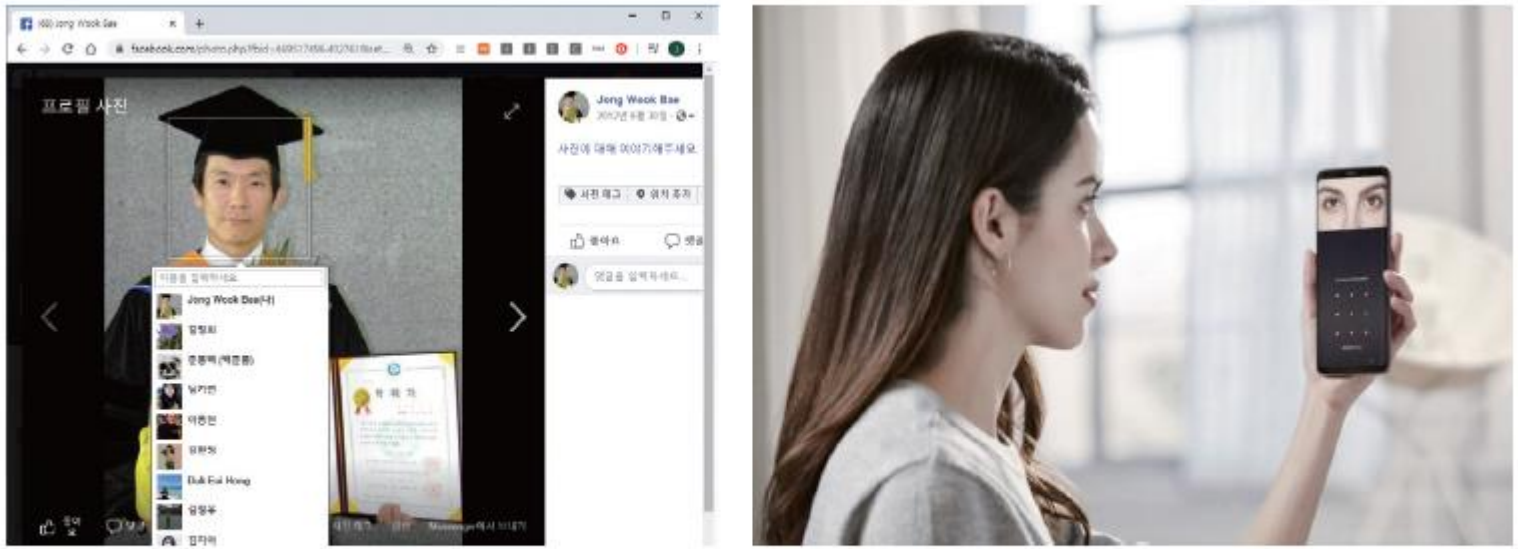
- ◆11.2.1 하르 기반 분류기
- ◆11.2.2 얼굴 검출 프로그램 구현
- ◆11.2.3 성별 분류 기초
- ◆11.2.4 얼굴 기울기 계산 및 보정
- ◆11.2.5 입술 영역 및 머리 영역 검출
- ◆11.2.6 히스토그램 비교
- ◆11.2.7 성별 분류

11.2 하르 분류기를 이용한 얼굴검출 및 성별 분류

◆얼굴 검출 및 인식 응용

❖ 저가형 디지털 카메라에서도 얼굴 검출 후 액정에 표시

❖ 페이스북이나 구글에 사진을 올리면 얼굴 영역 검출



참조: https://cdnimg.rg.ru

〈그림 11.2.1〉 얼굴 검출 응용 예

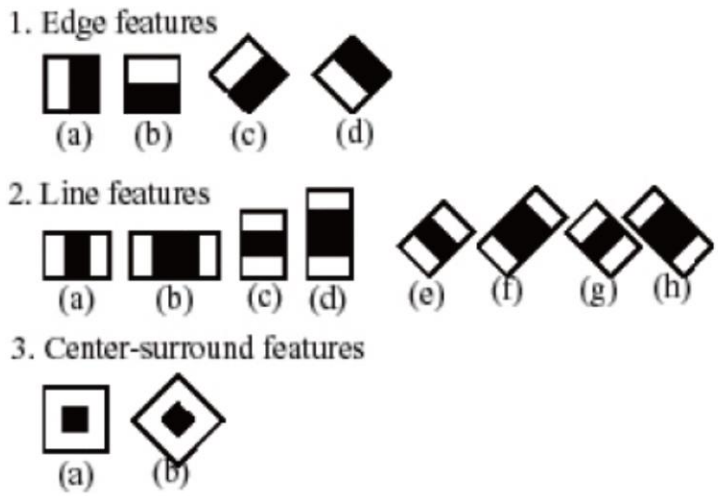
11.2.1 하르 기반 분류기

◆논문 “Rapid Object Detection Using a Boosted Cascade of Simple Features”

❖ 얼굴과 얼굴이 아닌 것의 차이를 효율적으로 보여줄 수 있는 하르 유사 특징(Haar-like features)을 이용한 방법 제안

❖ 특징값 정의

- 흰색 영역의 화소값 합
- 검은색 영역의 화소값 합의 차



〈그림 11.2.2〉 유사 하르 특징

11.2.1 하르 기반 분류기

◆하르 기반 캐스케이드 분류기

- ❖ 여러 개의 검출기를 순차적으로 사용
- ❖ 처음에 간단한 검출기를 적용하고, 진행할수록 복잡한 검출기 적용
- ❖ 단순 검출기를 통과한 후보에만 시간이 많이 걸리는 강력한 검출기가 적용
- ❖ → 검출 속도 크게 향상

◆OpenCV의 캐스케이드 분류기

- ❖ 1,000개 이상의 얼굴 영상과 10,000개 이상의 얼굴이 아닌 영상을 사용하여 학습

11.2.1 하르 기반 분류기

◆분류기 파일 사이트

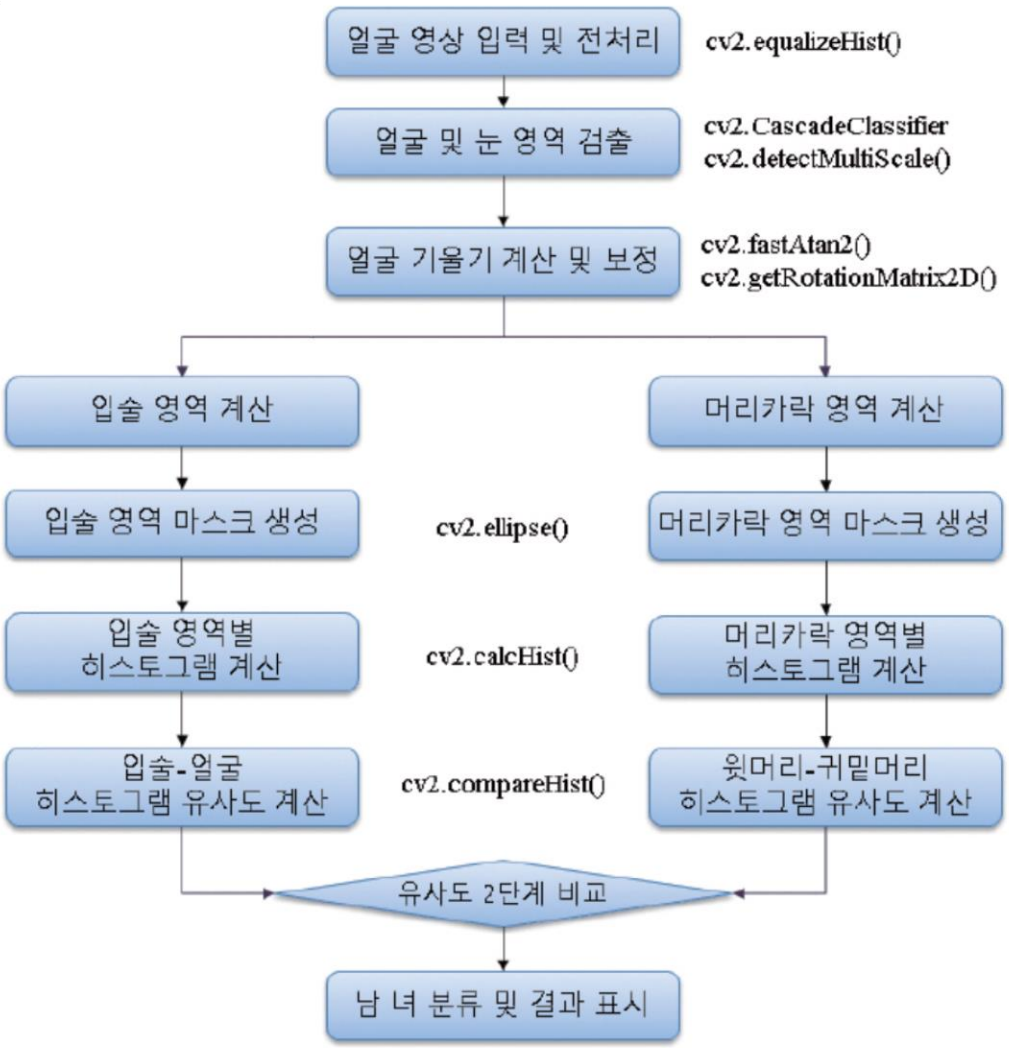
❖ https://github.com/opencv/opencv/tree/master/data/haarcascades

〈표 11.2.1〉 XML 검출기 목록

cascade classifer 분류기	XML 파일명
Face detector (default)	haarcascade_frontalface_default.xml
Face detector (fast Harr)	haarcascade_frontalface_alt2.xml
Face detector (fast LBP)	lbpcascade_frontalface.xml
Eye detector	haarcascade_eye.xml
Eye detector (separate for left)	haarcascade_lefteye_2splits.xml
Eye detector (separate for right)	haarcascade_righteye_2splits.xml
Mouth detector	haarcascade_mcs_mouth.xml
Nose detector	haarcascade_mcs_nose.xml
Whole person detector	haarcascade_fullbody.xml

11.2.2 얼굴 검출 프로그램 구현

◆전체 프로그램 구성



〈그림 11.2.3〉 성별 분류 프로그램 전체 구성

11.2.2 얼굴 검출의 구현

예제 11.2.1 얼굴 검출 - 06.detect_face.py

```
01  import cv2, numpy as np
02
03  def preprocessing(no):                                  # 전처리 수행 함수
04      image = cv2.imread('images/face/%2d.jpg' %no, cv2.IMREAD_COLOR)
05      if image is None: return None, None
06      gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)      # 명암도 영상 변환
07      gray = cv2.equalizeHist(gray)                       # 히스토그램 평활화
08      return image, gray
09
10  face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_alt2.xml") # 정면 얼굴 검출기
11  eye_cascade = cv2.CascadeClassifier("haarcascade_eye.xml")                # 눈 검출기
12  image, gray = preprocessing(34)                                           # 전처리
13  if image is None: raise Exception("영상파일 읽기 에러")
14
```

원본 영상과 명암도 영상 반환

11.2.2 얼굴 검출의 구현

영상 확대 비율

이웃 후보 개수

최소 얼굴 크기

얼굴 검출되면

검출 사각형

눈 2개 검출되면

각 눈의 중심 좌표

```
15  faces = face_cascade.detectMultiScale(gray, 1.1, 2, 0, (100, 100))        # 얼굴 검출
16  if faces.any():                                                          # 얼굴 사각형 검출되면
17      x, y, w, h = faces[0]
18      face_image = image[y:y+h, x:x+w]                                     # 얼굴 영역 영상 가져오기
19      eyes = eye_cascade.detectMultiScale(face_image, 1.15, 7, 0, (25, 20))   # 눈 검출 수행
20      if len(eyes) == 2:                                                   # 눈 사각형이 검출되면
21          for ex, ey, ew, eh in eyes:
                center = (x + ex + ew//2, y + ey + eh//2)                    # 중심점 계산
23              cv2.circle(image, center, 10, (0, 255, 0), 2)                # 눈 중심에 원 그리기
24      else:
25          print("눈 미검출")
26
27      cv2.rectangle(image, faces[0], (255, 0, 0), 2)        # 얼굴 검출 사각형 그리기
28      cv2.imshow("image", image)
29  else:
30      print("얼굴 미검출")
31  cv2.waitKey()
```

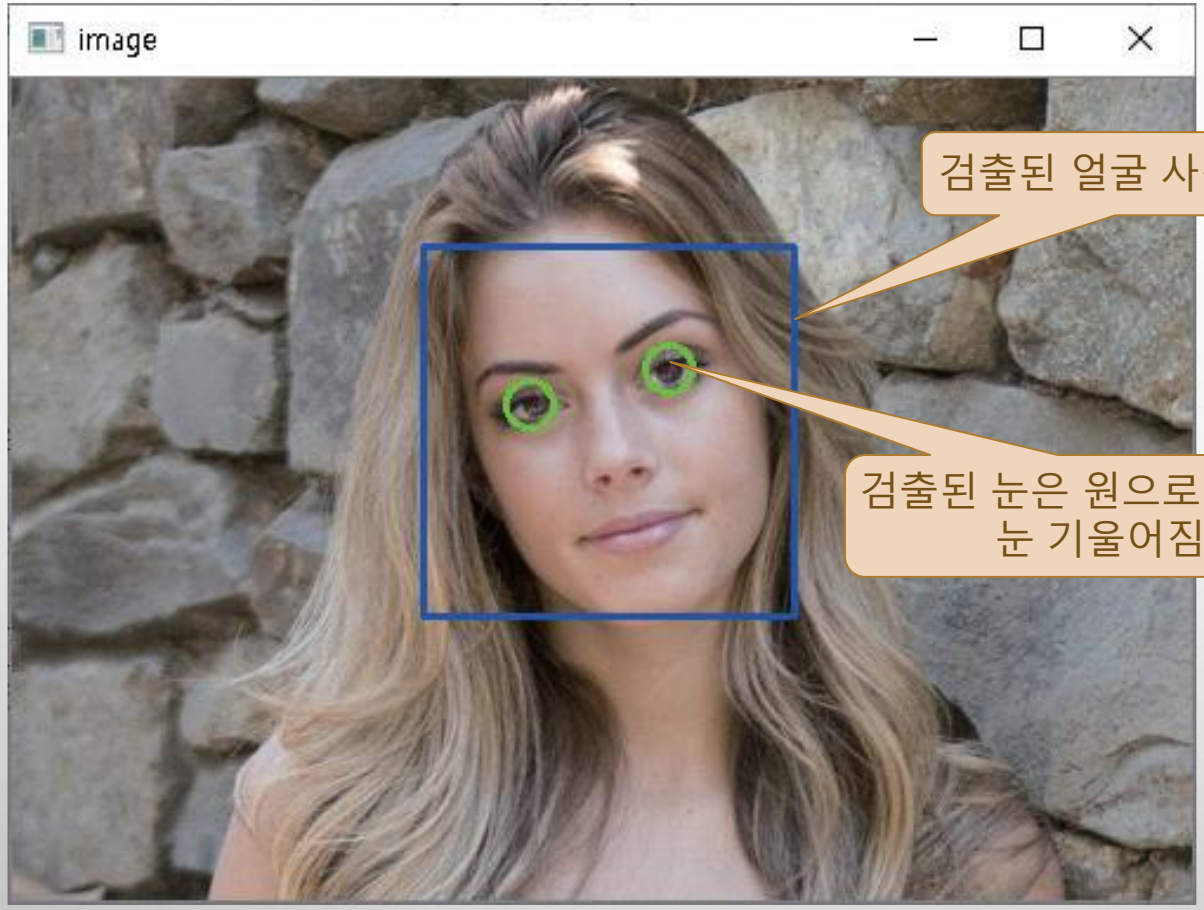
11.2.2 얼굴 검출의 구현

◆분류 수행 : CascadeClassifier::detectMultiScale() 함수 사용

함수 인수와 반환자료형 구조	
void CascadeClassifier::detectMultiScale(InputArray image, CV_OUT vector<Rect>& objects, double scaleFactor = 1.1, int minNeighbors = 3, int flags = 0, Size minSize = Size(), Size maxSize = Size());	
인수	설명
InputArray image	객체 검출 대상 행렬 (8비트 명암도 영상)
vector<Rect>& objects	반환되는 검출 객체 사각형
double scaleFactor	영상 크기 감소에 대한 규정
int minNeighbors	이웃 후보 사각형의 개수
int flags = 0	과거 함수에서 사용하던 flag
Size minSize	가능한 객체 최소 크기 – 이보다 작은 객체 무시
Size maxSize	가능한 객체 최대 크기 – 이보다 큰 객체 무시

11.2.2 얼굴 검출 프로그램 구현

◆실행결과



검출된 얼굴 사각형 표시

검출된 눈은 원으로 표시 → 눈 기울어짐

11.2.3 성별 분류 기초

◆여자와 남자의 구분에 대해서 두 가지 가정

1. 입술의 색깔이 남자에 비해 여자가 더 붉다
2. 머리카락의 길이가 남자에 비해서 여자가 더 길다.

❖ 이 가정이 정답은 아님

- 여성이라도 머리카락이 짧을 수 있음 / 남자라도 머리카락이 길 수 있음
- 남자가 입술 화장을 할 수도 있으며, 입술 화장을 하지 않는 여자도 있음

◆두 가지 가정을 두는 것

❖ 간단하면서도 쉽게 남녀를 구분 하기 위해

11.2.3 성별 분류 기초

◆ 입술 영역으로 남녀를 어떻게 구분

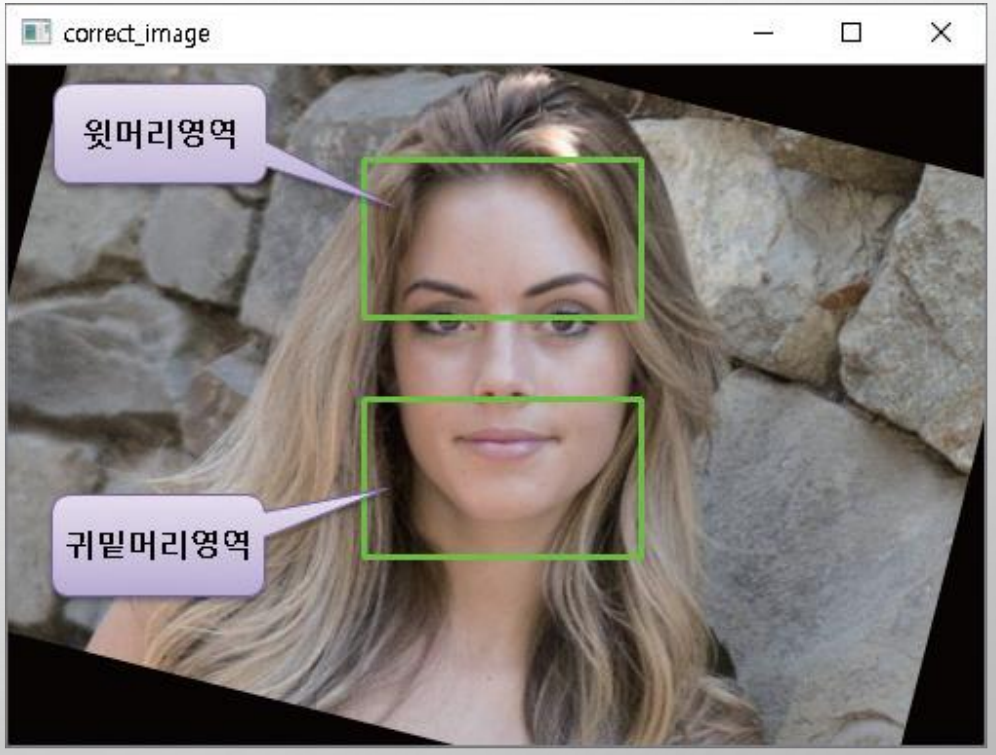
❖ 여자의 입술 색상이 더 붉다는 가정

- 입술 영역의 색상과 얼굴 영역의 색상을 비교
- →두 영역의 색상이 비슷하면 남자로 분류하고, 다르면 여자로 분류

❖ 여자의 긴 머리카락과 남자의 짧은 머리카락을 어떻게 인식할까?

- 귀밑 영역에 머리카락이 있는지를 검사하는 것

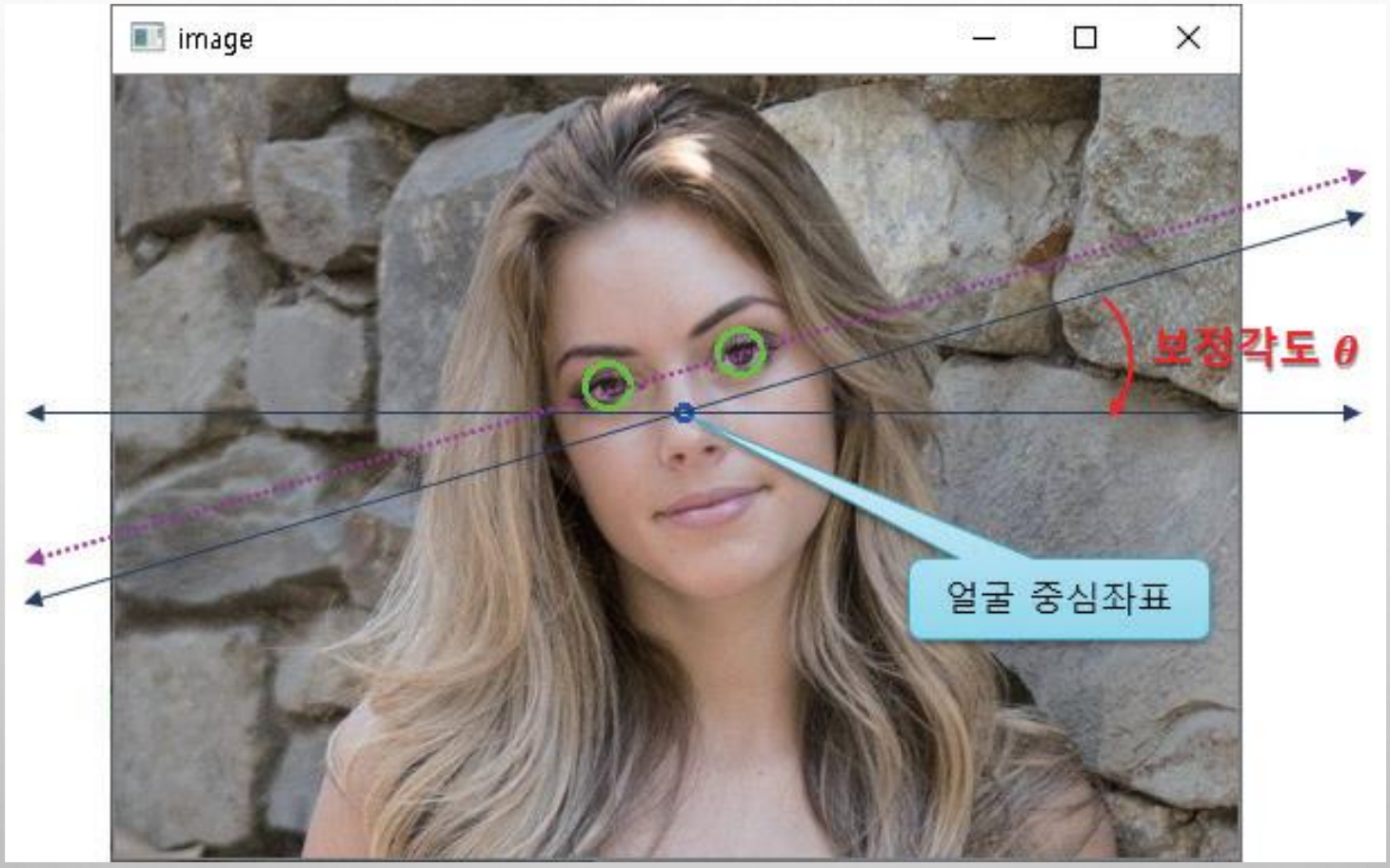
❖ cv2.compareHist() 함수로 유사도 비교



11.2.4 얼굴 기울기 계산 및 보정

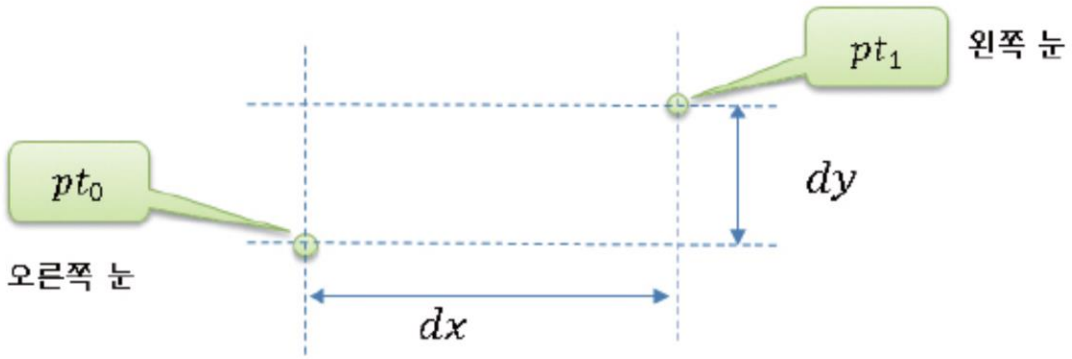
◆얼굴의 기울어진 각도는 두 눈의 중심좌표 이용

❖ 눈 검출 필요 , 중심점에서 기울기만큼 회전



11.2.4 얼굴 기울기 계산 및 보정

❖ 두 눈 좌표의 차분 좌표으로 역탄젠트를 취하면 기울기 계산 가능



$$(dx,\ dy) = pt_1 - pt_0$$

$$\theta = \tan^{-1}\left(\frac{dy}{dx}\right)$$

〈그림 11.2.6〉 검출된 두 눈을 이용한 기울기 보정 방법

11.2.4 얼굴 기울기 계산 및 보정

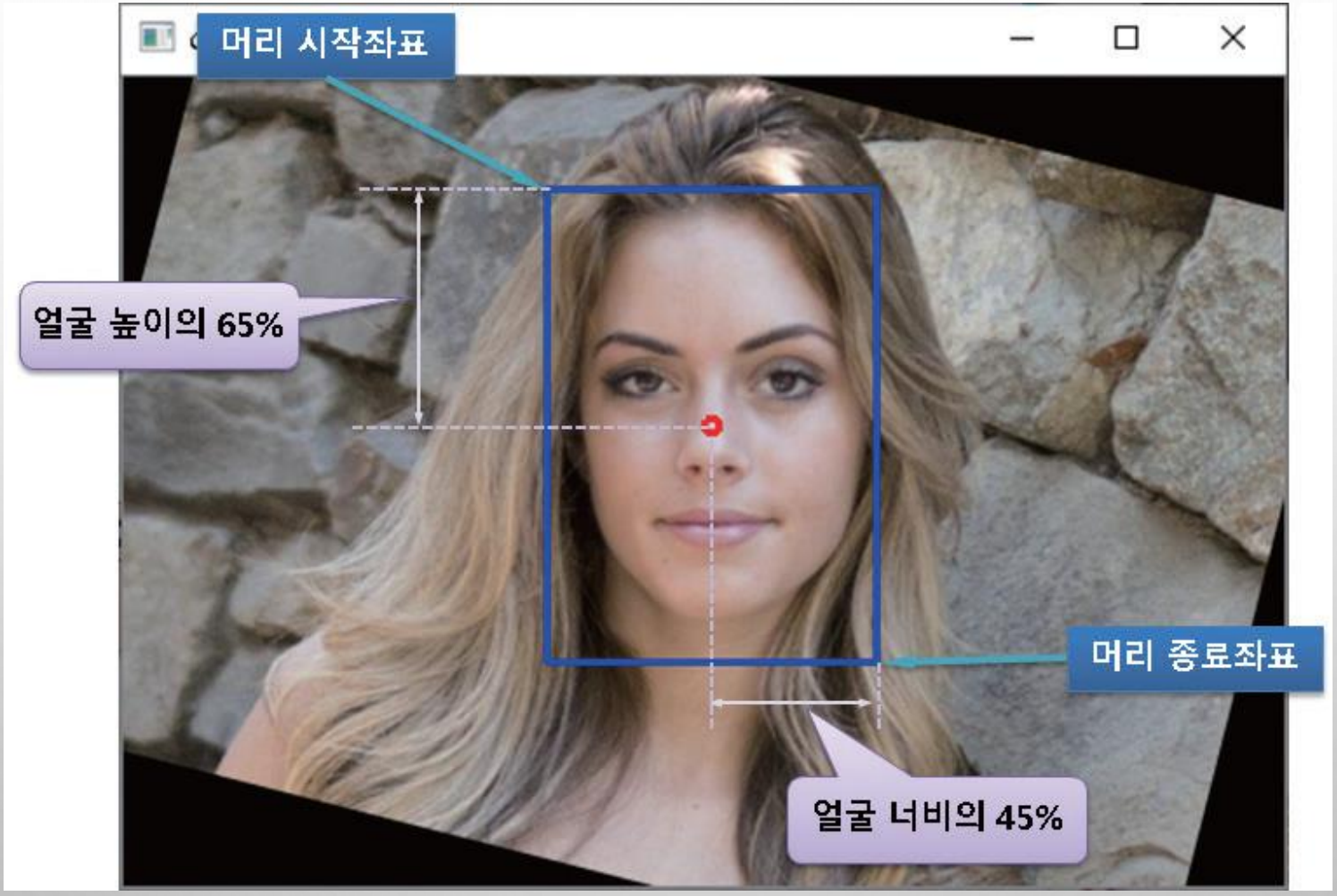
◆ ‘header/utils.py’ 파일 중에

```
def correct_image(image, face_center, eye_centers):
    pt0, pt1 = eye_centers                                  # 좌, 우 눈 중심 좌표
    if pt0[0] > pt1[0]: pt0, pt1 = pt1, pt0                 # 좌, 우 눈 위치 맞바꿈

    dx, dy = np.subtract(pt1, pt0).astype(float)            # 두 좌표간 차분 계산
    angle = cv2.fastAtan2(dy, dx)                           # 차분으로 기울기 계산
    rot_mat = cv2.getRotationMatrix2D(face_center, angle, 1)

    size = image.shape[1::-1]                               # 행태와 크기는 역순
    corr_image = cv2.warpAffine(image, rot_mat, size, cv2.INTER_CUBIC)

    eye_centers = np.expand_dims(eye_centers, axis=0)       # 눈 좌표 차원 증가

    corr_centers = cv2.transform(eye_centers, rot_mat)      # 어파인(회전) 변환 좌표 계산
    corr_centers = np.squeeze(corr_centers, axis=0)         # 보정 좌표 차원 감소

    return corr_image, corr_centers                         # 보정 결과 반환
```

수정

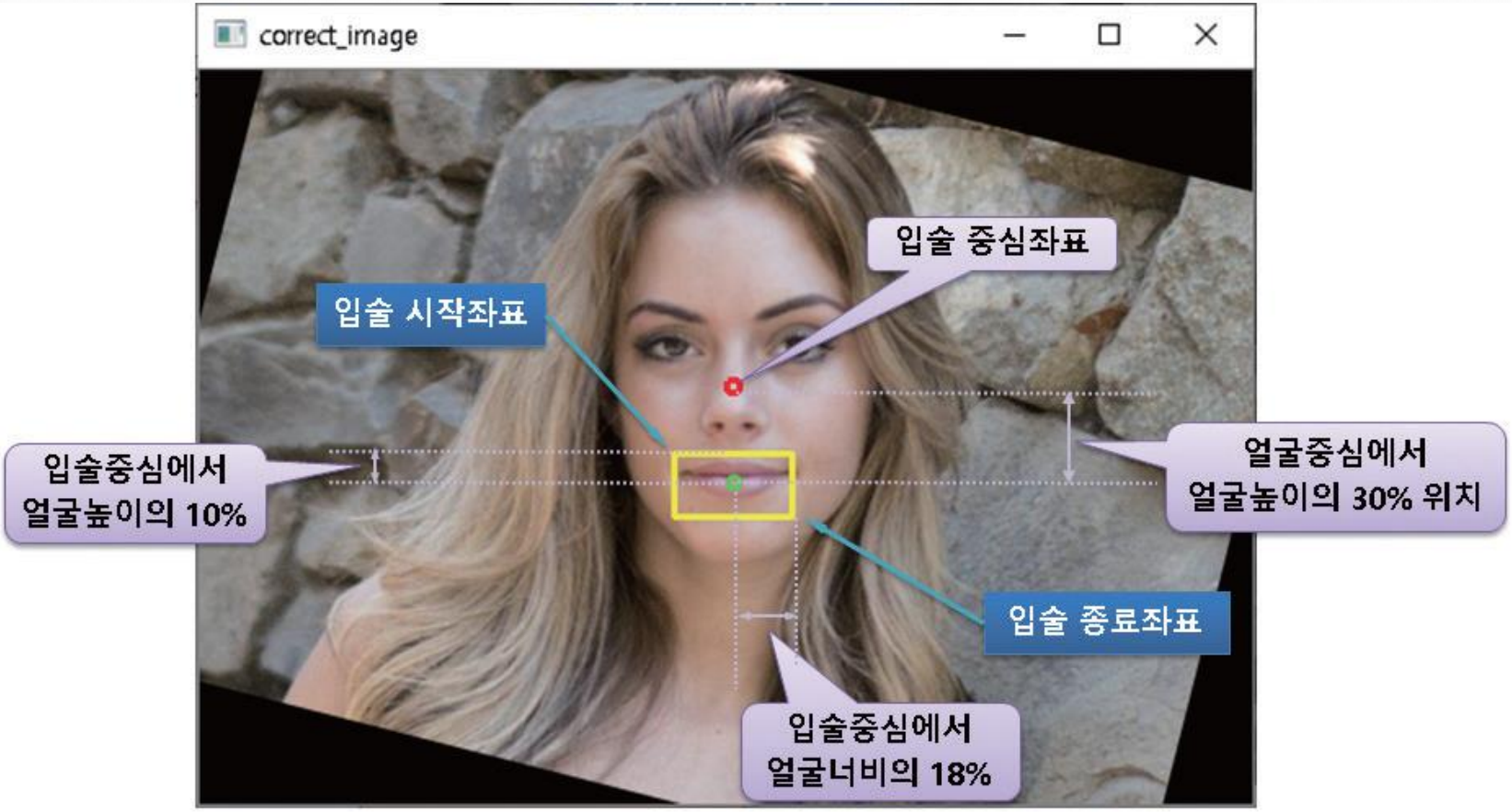
11.2.5 입술 영역 및 머리 영역 검출

❖ 윗머리 및 귀밑머리 영역 계산



11.2.5 입술 영역 및 머리 영역 검출

❖ 입술 영역 계산



11.2.5 입술 영역 및 머리 영역 검출

```
def define_roi(pt, size):
    return np.ravel((pt, size)).astype('int')          # 2 원소 튜플 2개 → 4 원소 튜플 1개

def detect_object(center, face):
    w, h = np.array(face[2:4])                         # 얼굴 영역 크기(w, h)
    center = np.array(center)                          # 얼굴 중심좌표 ndarray 객체로 변경
    gap1 = np.multiply((w, h) * (0.45, 0.65))          # 얼굴 영역 비율 크기 45%, 65%
    gap2 = np.multiply((w, h) * (0.18, 0.1))           # 입술 영역 비율 크기18%, 10%

    pt1 = center - gap1                                # 좌상단 평행이동– 머리 시작 좌표
    pt2 = center + gap1                                # 우하단 평행이동– 머리 종료 좌표
    hair = define_roi(pt1, pt2 - pt1)                  # 전체 머리 영역

    size = np.multiply(hair[2:4], (1, 0.4))            # 머리카락 영역 높이 40%
    hair1 = define_roi(pt1, size)                      # 윗머리 영역 (x, y, w, h)
    hair2 = define_roi(pt2-size, size)                 # 귀밑머리 영역

    lip_center = center + (0, int(h*0.3))              # 입술 영역 중심 좌표 – 30%
    lip1 = lip_center - gap2                           # 좌상단 평행이동
    lip2 = lip_center + gap2                           # 우하단 평행이동
    lip = define_roi(lip1, lip2 - lip1)                # 입술 영역

    return [hair1, hair2, lip, hair]                   # 각 영역을 리스트 구성 후 반환
```

두 좌표 차분 → 크기

두 좌표 차분 → 크기

입술영역 시작 좌표

머리영역 종료좌표

11.2.5 입술 영역 및 머리 영역 검출

◆중간 점검 프로그램 – 헤더 파일

예제 11.2.2 header/harr_utils.py

```
import numpy as np, cv2

def preprocessing(no): …

def correct_image(image, face_center, eye_centers): …

def define_roi(pt, size): …

def detect_object(center, face): …
```

◆중간 점검 프로그램 – 메인

오타

오타

예제 11.2.2 머리카락 및 입술 검출 – 07.detect_hair_lip.py

```
from haar_utils import *                                  # 전처리 및 영역 검출 함수 임포트

face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_alt2.xml") # 얼굴 검출기
eye_cascade = cv2.CascadeClassifier("haarcascade_eye.xml")       # 눈 검출기
image, gray = preprocessing(34)                                  # 전처리
if image is None: raise Exception("영상파일 읽기 에러")

faces = face_cascade.detectMultiScale(gray, 1.1, 2, 0, (100, 100))        # 얼굴 검출
```

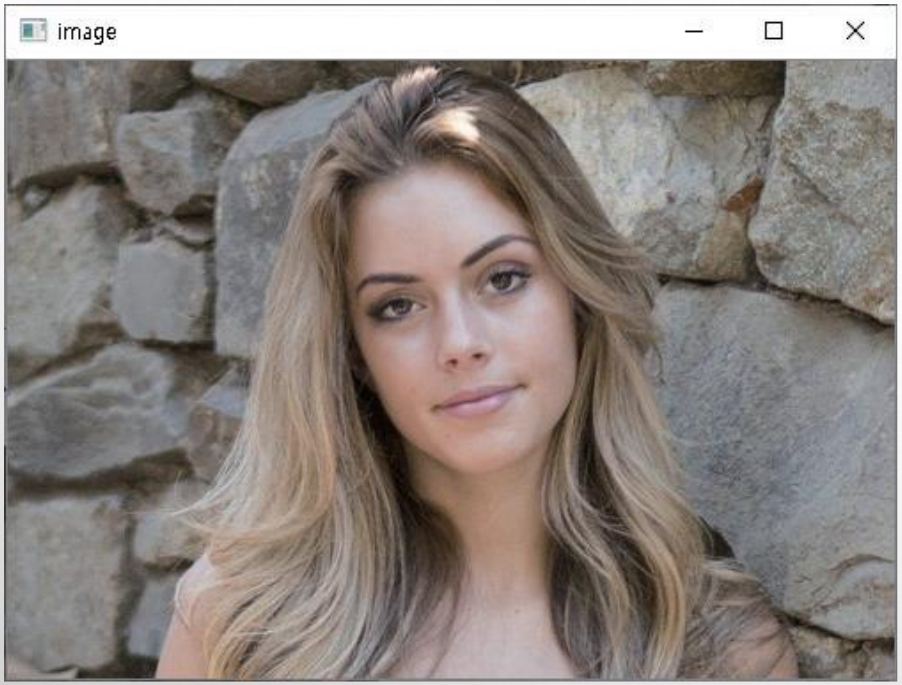
11.2.5 입술 영역 및 머리 영역 검출

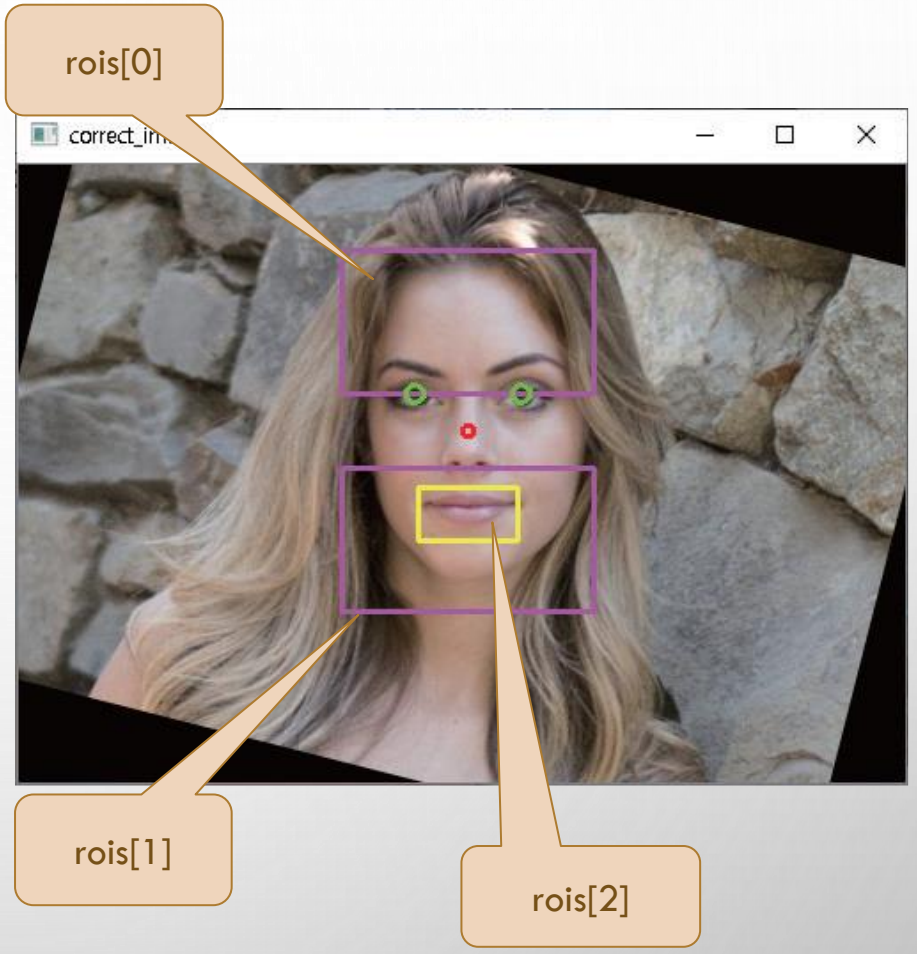
```
09 if faces.any() :
10     x, y, w, h = faces[0]
11     face_image = image[y:y+h, x:x+w]                    # 얼굴 영역 영상 가져오기
12     eyes = eye_cascade.detectMultiScale(face_image, 1.15, 7, 0, (25, 20)) # 눈 검출
13
14     if len(eyes) == 2:
15         face_center = (x+w//2, y+h//2)
16         eye_centers = [(x+ex+ew//2, y+ey+eh//2) for ex, ey, ew, eh in eyes]
17         corr_image, corr_center = correct_image(image, face_center, eye_centers) # 회전 보정
18         rois = detect_object(face_center, faces[0])          # 머리카락/입술 영역 계산
19
20         cv2.rectangle(corr_image, rois[0], (255, 0, 255), 2)    # 윗머리 영역
21         cv2.rectangle(corr_image, rois[1], (255, 0, 255), 2)    # 귀밑머리 영역
22         cv2.rectangle(corr_image, rois[2], (255, 0, 0), 2)      # 입술 영역
23
24         cv2.circle(corr_image, tuple(corr_center[0]), 5, (0, 255, 0), 2)    # 보정 눈 좌표
25         cv2.circle(corr_image, tuple(corr_center[1]), 5, (0, 255, 0), 2)    # 보정 눈 좌표
26         cv2.circle(corr_image, face_center, 3, (0, 0, 255), 2) # 얼굴 중심좌표
27         cv2.imshow("correct_image", corr_image)
28     else:
29         print("눈 미검출")
30 else:
31     print("얼굴 미검출")
32 cv2.imshow("image", image)
33 cv2.waitKey(0)
```

검출 영역 그리기

11.2.5 입술 영역 및 머리 영역 검출

◆실행결과

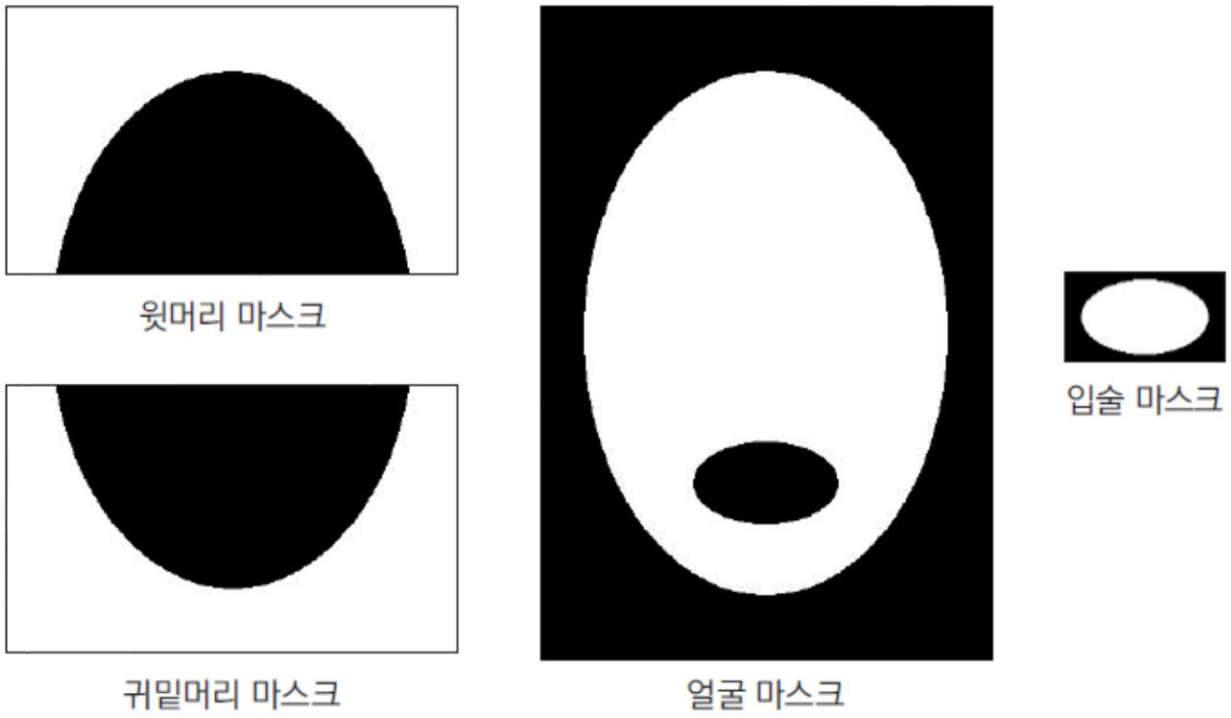




11.2.6 히스토그램 비교

❖ 히스토그램 산출과정에서 각 영상에 해당하는 부분만 계산 → 마스크 이용

- 입술영역 히스토그램은 입술영역만
- 얼굴 영역 히스토그램은 얼굴 영역만 계산
- 윗머리/ 귀밑머리도 그 영역만 계산



〈그림 11.2.9〉 얼굴 영상 히스토그램을 위한 마스크 영상의 예

11.2.6 히스토그램 비교

◆타원 그리기 함수 구현 (‘header/haar_histogram.py’ 중)

```
def draw_ellipse(image, roi, ratio, color, thickness=cv2.FILLED):
    x, y, w, h = roi
    center = (x + w//2, y + h//2)                                  # 타원 중
    size = (int(w*ratio), int(h*ratio))                            # 타원 크
    cv2.ellipse(image, center, size, 0, 0, 360, color, thickness)  # 0~360도 타원
    return image
```

그려질 타원 비율

전체머리영역의 45%를 타원 반지름으로 사용

◆각 마스크 생성 함수(‘header/haar_histogram.py’ 중)

```
def make_masks(rois, shape):                                  # 영역별 마스크 생성 함수
    base_mask = np.full(shape, 255, np.uint8)
    hair_mask = draw_ellipse(base_mask, rois[3], 0.45, 0)     # 얼굴 타원 그리기
    lip_mask = draw_ellipse(np.copy(base_mask), rois[2], 0.40, 255)  # 입력 타원 그리기

    masks = [hair_mask, hair_mask, lip_mask, ~lip_mask]       # 4개 마스크 구성
    masks = [mask[y:y+h,x:x+w] for mask, (x, y, w, h) in zip(masks, rois)]

    # for i, mask in enumerate(masks):                        # 마스크 영상 윈도우 표시
    #         cv2.imshow('mask'+str(i), mask)
    # cv2.waitKey()

    return masks
```

기본 마스크에 입술영역 타원 그림

입술 마스크 반전하여 구성

11.2.6 히스토그램 비교

❖ 마스크 이용하여 각 서브영역의 히스토그램 생성

- 기울기 보정 영상에 각 서브 영역 및 마스크로 히스토그램 계산
- ('header/haar_histogram.py' 중)

각 영역 히스토그램 반환

해당 영역 참조영상

각 영역 마스크 4개

```
def calc_histo(image, rois, masks):
    bsize = (64, 64, 64)                                    # 히스토그램 계급 개수
    ranges = (0,256, 0,256, 0,256)                          # 각 채널 빈도 범위
    subs = [image[y:y+h, x:x+w] for x, y, w, h in rois]     # 관심 영역 참조로 영상 생성

    hists = [cv2.calcHist([sub], [0, 1, 2], mask, bsize, ranges, 3)
                    for sub, mask in zip(subs, masks)]      # 관심 영역 영상 히스토그램
    hists = [h / np.sum(h) for h in hists]                  # 히스토그램값 정규화
    sim1= cv2.compareHist(hists[2], hists[3], cv2.HISTCMP_CORREL)   # 입술-얼굴 유사도
    sim2= cv2.compareHist(hists[0], hists[1], cv2.HISTCMP_CORREL)   # 윗-귀밑머리 유사도
    return sim1, sim2
```

11.2.6 히스토그램 비교

◆헤더 파일 추가

예제 11.2.3 히스토그램 관련 헤더 파일 – header/harr_histogram.py

```
01  import numpy as np, cv2
02
03  def draw_ellipse(image, roi, color, ratio): …
11
12  def make_masks(sub, org_size): …
21
22  def calc_histo(image, rois, masks): …
```

예제 11.2.3 영역별 유사도 계산 – 08.compare_hists.py

```
01  from haar_utils import *                                  # 분류기 적재 및 전처리 함수
02  from haar_histogram import *                              # 히스토그램 비교 관련 함수
03
04  face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_alt2.xml") # 얼굴 분류기
05  eye_cascade = cv2.CascadeClassifier("haarcascade_eye.xml")        # 눈 분류기
06  image, gray = preprocessing(34)                                   # 전처리
07  if image is None: raise Exception("영상파일 읽기 에러")
08
09  faces = face_cascade.detectMultiScale(gray, 1.1, 2, 0, (100, 100))        # 얼굴 검출
```

11.2.6 히스토그램 비교

```
10  if faces.any():
11      x, y, w, h = faces[0]
12      face_image = image[y:y+h, x:x+w]                    # 얼굴 영역 영상 가져오기
13      eyes = eye_cascade.detectMultiScale(face_image, 1.15, 7, 0, (25, 20)) # 눈 검출
14
15      if len(eyes) == 2:
16          face_center = (x + w//2, y + h//2)
17          eye_centers  = [(x+ex+ew//2, y+ey+eh//2) for ex, ey, ew, eh in eyes]
18          corr_image, corr_center = correct_image(image, face_center, eye_centers)
19
20          rois = detect_object(face_center, faces[0])          # 머리 및 입술영역 검출
21          masks = make_masks(rois, corr_image.shape[:2])       # 4개 마스크 생성
            sim = calc_histo(corr_image, rois, masks)            # 4개 히스토그램 생성

24          print("입술-얼굴 유사도: %4.2f" % sim[0])            # 결과 출력
25          print("윗-귀밑머리 유사도: %4.2f" % sim[1])
26      else:
27          print("눈 미검출")
28  else:
29      print("얼굴 미검출")
```

마스크 이용
히스토그램 및 유사 계산

윗버리 vs 귀밑머리 유사도

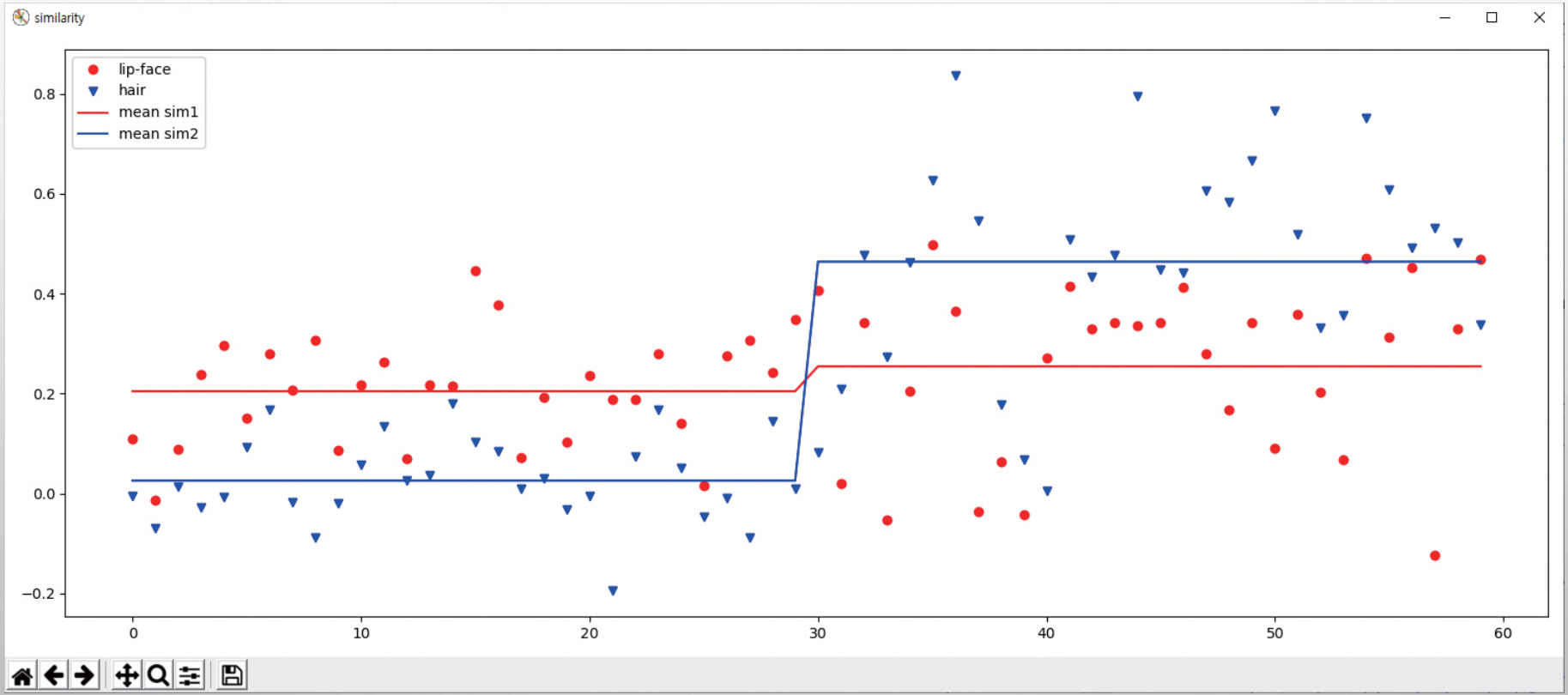
입술 vs 얼굴 유사도

11.2.6 히스토그램 비교

◆실행결과

```
Run: 08.compare_hist
C:\Python\python.exe D:/source/chap11/08.compare_hist.py
입술-얼굴 유사도: 0.21
윗-귀밑머리 유사도: 0.44
```

11.2.7 성별 분류


similarity
lip-face
hair
mean sim1
mean sim2
0.8
0.6
0.4
0.2
0.0
−0.2
0
10
20
30
40
50
60

11.2.7 성별 분류

◆윗머리와 귀밑머리의 유사도(criteria1)

❖ 크다 → 두 영역 색상 비슷

❖ →귀밑에 머리카락 있을 확률↑ → 여자확률↑

◆얼굴과 입술의 유사도(criteria2)

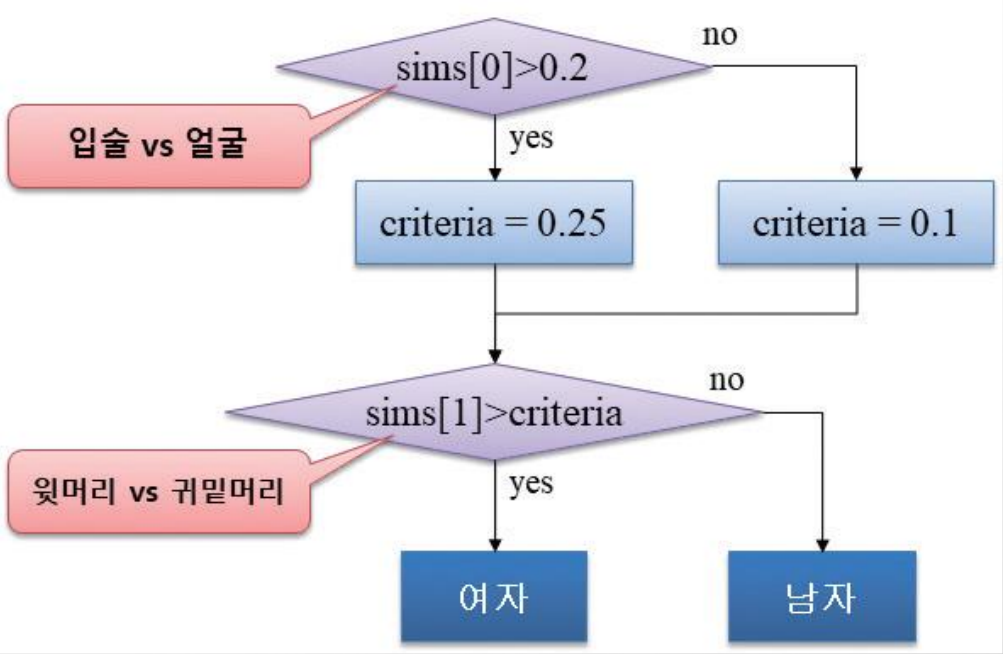
❖ 작다 → 입술과 얼굴 색상 다름

❖ → 입술색이 도르라짐 →여자 확률↑

◆두 가지 정보만으로 남녀 구분 어려움

❖ 최대한 좋은 결과를 도출하기 위한 방법

❖ 2번의 임계값 적용



11.2.7 성별 분류

◆헤더 파일에 추가

예제 11.2.4 분류 및 영상 출력 함수 – header/haar_classify.py

```
import cv2
from haar_histogram import draw_ellipse
from Common.utils import put_string

def classify(image, sim, no):
    sim = 0.25 if sims[0] > 0.2 else 0.1                    # 얼굴-입술 유사도
    text = 'Woman' if sims[1] > sim else 'Man'              # 윗머리-귀밑머리 유사도

    text = "%02dg: %s" %no, text                            # 분류 결과 출력
    result = "유사도[입술-얼굴: %4.2f 윗-귀밑머리: %4.2f]" % (sims)

    put_string(image, text, (10, 30), "")                   # 영상 출력
    print(text + " - " + result)                            # 실행창 출력

def display(image, face_center, centers, rois):
    cv2.circle(image, face_center, 3, (0, 0, 255), 2)       # 얼굴 중심점 표시
    cv2.circle(image, tuple(centers[0]), 10, (0, 255, 0), 2)  # 눈 표시
    cv2.circle(image, tuple(centers[1]), 10, (0, 255, 0), 2)
    draw_ellipse(image, rois[2], 0.45, (0, 0, 255), 2)      # 얼굴 타원
    draw_ellipse(image, rois[3], 0.45, (255, 100, 0), 2)    # 입술 타원
    cv2.imshow("correct_image", corr_image)
```

수정

11.2.7 성별 분류

◆최종 완성 프로그램

예제 11.2.5 성별 분류 프로그램 완성 – 09.genderClassifier.cpp

```
from haar_utils import *                                  # 검출기 적재 및 전처리 함수
from haar_histogram import *                              # 히스토그램 비교 관련 함수

face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_alt2.xml")
eye_cascade = cv2.CascadeClassifier("haarcascade_eye.xml")

no, max_no, cnt = 0, 60, 1
while True:
    no = no + cnt
    image, gray = preprocessing(no)                       # 전처리 수행
    if image is None:                                     # 영상 읽기 예외처리
        print("%02d.jpg: 영상파일 없음" % no)
        if no < 0 : no = max_no-1
        elif no >= max_no: no = 0
        continue

    faces = face_cascade.detectMultiScale(gray, 1.1, 2, 0, (100, 100))    # 얼굴 검출
    if faces.any():
        x, y, w, h = faces[0]
        face_image = image[y:y+h, x:x+w]                  # 얼굴 영역 영상 가져오기
```

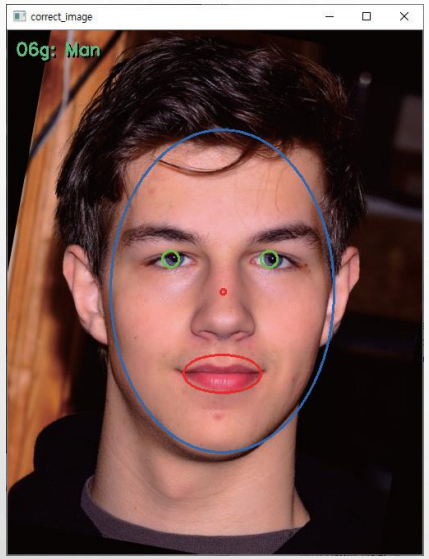
11.2.7 성별 분류

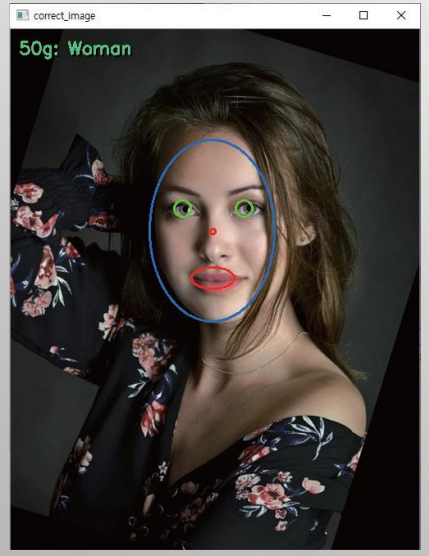
```
21            eyes = eye_cascade.detectMultiScale(face_image, 1.15, 7, 0, (25, 20))
22            if len(eyes) == 2:
23                face_center = (x+w//2, y+h//2)
24                eye_centers = [(x+ex+ew//2, y+ey+eh//2) for ex, ey, ew, eh in eyes]
25                corr_image, corr_centers = correct_image(image, face_center, eye_centers)
26
27                rois = detect_object(face_center, faces[0])        # 4개 영역 계산
28                masks = make_masks(sub_roi, corr_image.shape[:2]) # 각 영역 마스크 생성
29                sims = calc_histo(corr_image, rois, masks)         # 히스토그램 생성
30
31                classify(corr_image, sims, no)                     # 성별 분류 및 표시
32                display(corr_image, face_center, corr_centers, rois) # 얼굴, 눈 표시
33            else:
34                print("%02d.jpg: 눈 미검출" % no)
35        else:
36            print("%02d.jpg: 얼굴 미검출" % no)
37
38        key = cv2.waitKeyEx(0)                          # 키 이벤트 대기
39        if key == 2490368: cnt = 1                      # 위쪽 화살표 키이면 다음 영상
40        elif key == 2621440: cnt = -1                   # 아래쪽 화살표 키이면 이전 영상
41        elif key == 32 or key == 27: break              # 프로그램 종료 조건
```

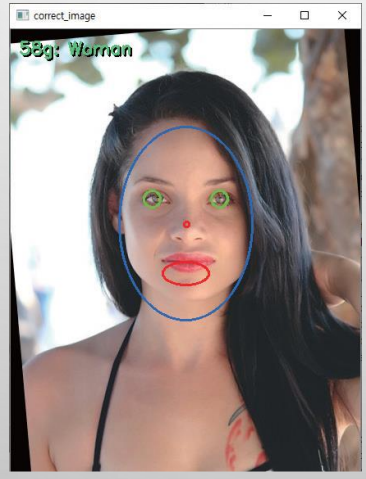
11.2.7 성별 분류

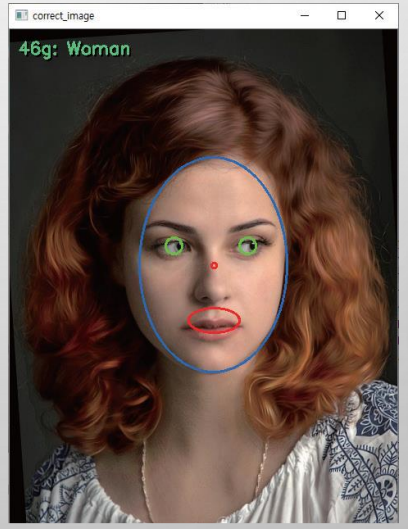
◆실행결과

❖ 화살표키로 다음 영상 분류 수행









단원 요약

◆1. 그림판 프로그램에서 팔레트를 이용해서 3차원의 색을 쉽게 표시하고 좌표를 클릭해서사용하기 위해서 색상은 HSV 컬러로 지정하고, BGR 컬러로 변환하여 표시한다.

◆2. 그림판 프로그램의 그리기 명령으로 드래그 상황에서 해당 개체가 움직이듯 그려지게 하려면 원본 영상이 아닌 복사본에 해당 개체를 그리면 된다.

◆3. cv2.calcHist() 함수는 1개 이상의 채널을 갖는 행렬에서 다차원의 히스토그램을 계산할 수 있다.

◆4. 색상과 채도의 2채널 행렬로 만든 2차원 히스토그램을 그래프로 표현하려면, 그래프 행렬에서 x 좌표는 채도를 y 좌표는 색상을 나타내도록 하고, 히스토그램 빈도값을 그 좌표에서의 밝기로 지정한다. 그러면 x, y 좌표로 색상과 채도를 인식할 수 있으며, 그 좌표의 밝기로 빈도의 많고 적음을 파악할 수 있다.

◆5. 두 개의 히스토그램을 비교하기 위해서 c v2.compareHist() 함 수를 사 용하며, C V_COMP_CORREL 옵션은 상호상관 함수의 수식을 응용한 것으로 두 히스토그램의 유사도를 계산한다.

단원 요약

◆6. 하르 기반 캐스케이드 분류기는 단순한 여러 개의 분류기를 순차적으로 사용하여 분류를 수행한다. 처음에 간단한 검출기를 적용하고, 진행할수록 복잡한 검출기를 적용한다. 단순 검출기를 통과한 후보에만 시간이 많이 걸리는 강력한 검출기가 적용되기 때문에 검출 속도를 빨리할 수 있다.

◆7. OpenCV에서 하르 기반 검출기는 얼굴, 눈, 코, 입 등을 검출하는 XML 파일들이 있다. 객체 검출을 위해서는 CascadeClassifier 클래스를 이용하며, 내부 메서드인 CascadeClassifier.load() 함수로 해당 XML 파일을 로드하고, CascadeClassifier. detectMultiScale() 함수로 검출을 수행한다.

◆8. 하르 검출기로 검출된 얼굴과 눈 영역으로 얼굴 중심좌표와 눈 중심좌표를 구할 수 있다. 그리고 두 눈의 중심좌표로 얼굴의 기울어진 각도를 계산할 수 있으며, 이 각도만큼 얼굴 중심좌표를 기준으로 회전하면 기울기를 보정할 수 있다.